Expt. No. 2

Title To implement McCulloch Pitts model for binary logic functions

Rubric	***Score (0 to 5)***
Knowledge	
Delivery	
Output and Conclusion	
Total	

Performed On: 07/08/2025

***Sign:*_____**

Experiment No. 2

Deep learning

Aim: Implement McCulloch Pitts model for binary logic function

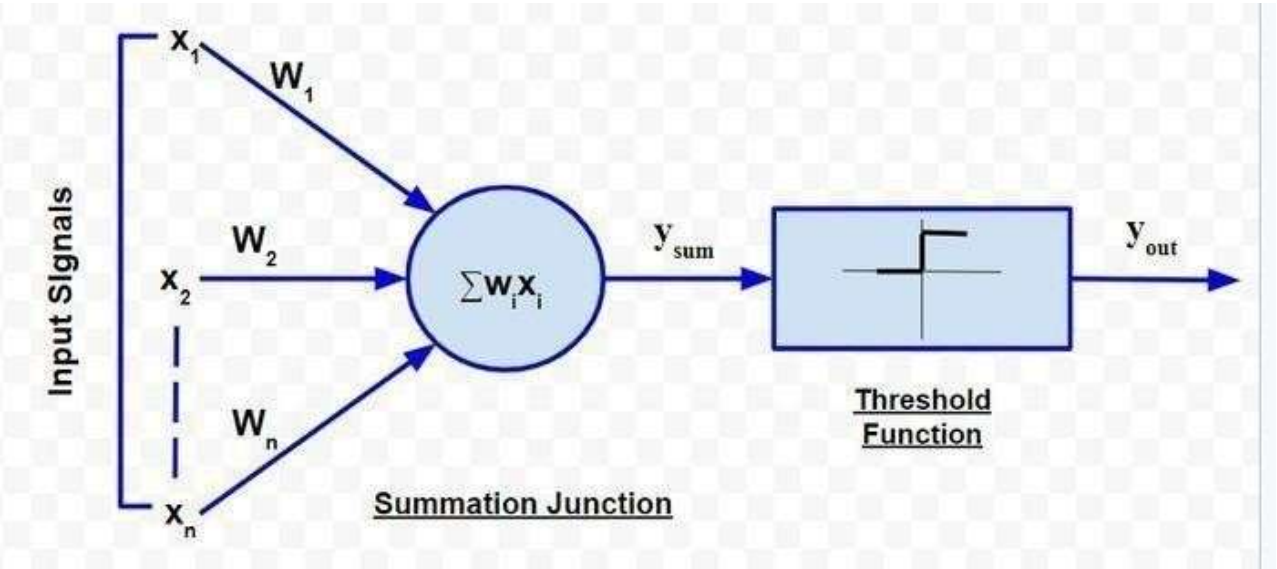
Software: Google collab

Pre-Lab Questions: Explain McCulloch and Pitts model?
The McCulloch-Pitts model is a simple artificial neuron that computes a weighted sum of inputs and produces a binary output using a threshold function.

Post-Lab Questions: Explain simulation of And gate using Mc Culloch -Pitts model
An AND gate in the McCulloch-Pitts model is simulated by assigning weights of 1 to each input and setting the threshold to 2 so output is 1 only if both inputs are 1.

Theory: The McCulloch-Pitts neural model, which was the earliest ANN model, has only two types of inputs — **Excitatory and Inhibitory.** The excitatory inputs have weights of positive magnitude, and the inhibitory weights have weights of negative magnitude. The inputs of the McCulloch-Pitts neuron could be either 0 or 1. It has a threshold function as an activation function. So, the output signal y_{out} is 1 if the input y_{sum} is greater than or equal to a given threshold value, else 0. The diagrammatic representation of the model is as follows

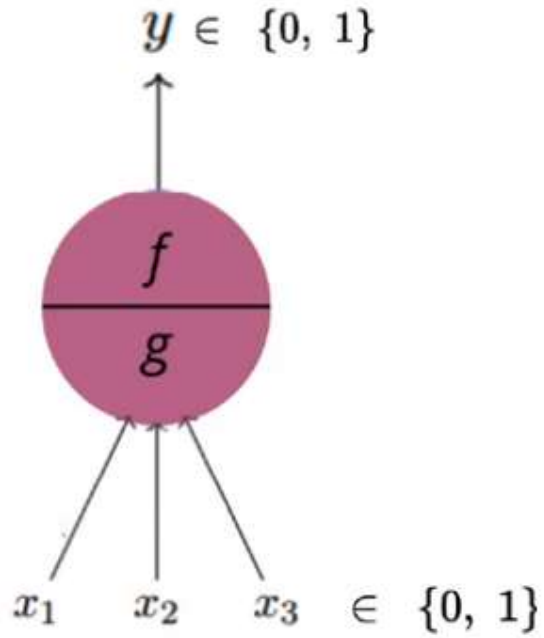
Simple McCulloch-Pitts neurons can be used to design logical operations. For that purpose, the connection weights need to be correctly decided along with the threshold function (rather than the threshold value of the activation function).



Simulation of And Gate using McCulloch Pitts Network

McCulloch-Pitts Neuron Model:

MP Neuron Model introduced by Warren McCulloch and Walter Pitts in 1943. MP neuron model is also known as linear threshold gate model.
Mathematical Model



Fig—2 Simple representation of MP Neuron Model

The function (soma) is actually split into two parts: **g**—The aggregates the inputs to a single numeric value and the function **f** produces the output of this neuron by taking the output of the **g** as the input i,e.. a single value as its argument. The function **f** will output the value 1 if the aggregation performed by the function **g** is greater than some threshold else it will return 0.
The inputs x1, x2, ….. xn for the MP Neuron can only take Boolean values and the inputs can be inhibitory or excitatory. Inhibitory inputs can have maximum effect on the decision-making process of the model. In some cases, inhibitory inputs can influence the final outcome of the model.

$$y = 0 \text{ if any } x_i \text{ is inhibitory, else}$$

$$g(x_1, x_2, ...x_n) = g(x) = \sum_{i=1}^{n} x_i$$

$$y = f(g(x)) = 1 \text{ if } g(x) \geq b$$

$$= 0 \text{ if } g(x) < b$$

Fig—3 Mathematical representation

For example, I can predict my own decision of whether I would like to watch a movie in a nearby IMAX theater tomorrow or not using an MP Neuron Model. All the inputs to the model are boolean i.e., [0,1] and

from the Fig—2 we can see that output from the model will also be boolean. (0—Not going to movie, 1—going to the movie)

Inputs for the above problem could be

- x1—IsRainingTomorrow (Whether it's going to rain tomorrow or not)
- x2—IsScifiMovie (I like science fiction movies)
- x3—IsSickTomorrow (Whether I am going to be sick tomorrow or not depends on any symptoms, eg: fever)
- x4—IsDirectedByNolan (Movie directed by Christopher Nolan or not.) etc….

In this scenario, if x3—IsSickTomorrow is equal to 1, then the output will always be 0. If I am not feeling well on the day of the movie then no matter whoever is the actor or director of the movie, I wouldn't be going for a movie.

Loss Function

Let's take an example of buying a phone based on some features of the features in the binary format. { y—0: Not buying a phone and y—1: buying a phone}



Launch (within 6 months)	0	1	1	0	0	1	0	1	1
Weight (<160g)	1	0	1	0	0	0	1	0	0
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1
dual sim	1	1	0	0	0	1	0	1	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1
NFC	0	1	1	0	1	0	1	1	1
Radio	1	0	0	1	1	1	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0
Price > 20k	0	1	1	0	0	0	1	1	1
Like (y)	1	0	1	0	1	1	0	1	0

Fig—4: Buying a phone

For each particular phone (observation) with a certain threshold value **b**, using the MP Neuron Model, we can predict the outcome using a condition that the summation of the inputs is greater than b then the predicted value will be 1 or else it will be 0. The loss for the particular observation will be squared difference between the Yactual and Ypredicted.

$$\hat{y} = \sum_{i=1}^{n} x_i \geq b$$

Fig—5: MP Neuron Model for Buying a phone

Similarly, for all the observations, calculate the summation of the squared difference between the Yactual and Ypredicted to get the total loss of the model for a particular threshold value **b**.

$$loss = \sum_i (y_i - \hat{y}_i)^2$$

Fig—6: Loss of the Model

Algorithm:

- The purpose of the learning algorithm is to find out the best value for the parameter **b** so that the loss of the model will be minimum. In the ideal scenario, the loss of the model for the best value of **b** would be zero.
- For n features in the data, the summation we are computing in Fig—5 can take only values between 0 and n because all of our inputs are binary (0 or 1). 0—indicates all the features described in the Fig—4 are off and 1—indicates all the features are on. Therefore the different values the threshold b can take will also vary from 0 to n. As we have only one parameter with a range of values 0 to n, we can use the brute force approach to find the best value of b.
- Initialize the b with a random integer [0,n] For each observation Find the predicted outcome, by using the formula in Fig—5 Calculate the summation of inputs and check whether its greater than or equal to b. If its greater than or equal to b, then the predicted outcome will be 1 or else it will be 0.
- After finding the predicting outcome compute the loss for each observation. Finally, compute the total loss of the model by summing up all the individual losses. Similarly, we can iterate over all the possible values of b and find the total loss of the model. Then we can choose the value of b, such that the loss is minimum.
- Model Evaluation
- After finding the best threshold value b from the learning algorithm, we can evaluate the model on the test data by comparing the predicted outcome and the actual outcome.

Training data

Launch (within 6 months)	0	1	1	0	0	1	0	1	1	0
Weight (<160g)	1	0	1	0	0	0	1	0	0	1
Screen size (<5.9 in)	1	0	1	0	1	0	1	0	1	0
dual sim	1	1	0	0	0	1	0	1	0	0
Internal memory (>= 64 GB, 4GB RAM)	1	1	1	1	1	1	1	1	1	0
NFC	0	1	1	0	1	0	1	1	1	0
Radio	1	0	0	1	1	1	0	0	0	0
Battery(>3500mAh)	0	0	0	1	0	1	0	1	0	0
Price > 20k	0	1	1	0	0	0	1	1	1	0
Like? (y)	1	1	1	0	0	1	1	1	0	0
predicted	1	1	0	1	1	1	1	0	0	0

Test data

1	0	0	1
0	1	1	1
0	1	1	1
0	1	0	0
1	0	0	0
0	0	1	0
1	1	1	0
1	1	1	0
0	0	1	0
0	1	0	0
0	1	1	0

Fig—7: Predictions on the test data for b = 5

For evaluation, we will calculate the accuracy score of the model.

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

Fig—8: Accuracy metric

For the above-shown test data, the accuracy of the MP neuron model = 75%.

Program & Output

```
import numpy as np

# McCulloch-Pitts Model
class McCullochPittsNeuron:
    def __init__(self, weights, threshold):
        # Initialize with weights and threshold
        self.weights = weights
        self.threshold = threshold

    def activate(self, inputs):
        """
        Activates the neuron using the binary step function.
        inputs: list of inputs
        Returns: 1 if weighted sum is greater than threshold, otherwise 0
        """
        # Weighted sum of inputs
        weighted_sum = np.dot(inputs, self.weights)

        # threshold
        return 1 if weighted_sum >= self.threshold else 0

def AND(inputs):
    weights = [1, 1]  # AND function needs positive weights
    threshold = 2  # Threshold set to 2 for AND
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)

def OR(inputs):
    weights = [1, 1]  # OR function needs positive weights
    threshold = 1  # Threshold set to 1 for OR
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)

def NOT(inputs):
    weights = [-1]  # NOT function uses negative weights
    threshold = -0.5  # Threshold set to -0.5 for NOT
    neuron = McCullochPittsNeuron(weights, threshold)
    return neuron.activate(inputs)
```

```
# AND Gate
print("AND Gate")
print("0 AND 0:", AND([0, 0]))
print("0 AND 1:", AND([0, 1]))
print("1 AND 0:", AND([1, 0]))
print("1 AND 1:", AND([1, 1]))

# OR Gate
print("\nOR Gate")
print("0 OR 0:", OR([0, 0]))
print("0 OR 1:", OR([0, 1]))
print("1 OR 0:", OR([1, 0]))
print("1 OR 1:", OR([1, 1]))

# NOT Gate
print("\nNOT Gate")
print("NOT 0:", NOT([0]))
print("NOT 1:", NOT([1]))
```

OUTPUT:

```
AND Gate
0 AND 0: 0
0 AND 1: 0
1 AND 0: 0
1 AND 1: 1

OR Gate
0 OR 0: 0
0 OR 1: 1
1 OR 0: 1
1 OR 1: 1

NOT Gate
NOT 0: 1
NOT 1: 0
```

Conclusion:

The McCulloch-Pitts model provides a simple framework for implementing binary logic functions by using weighted inputs and a threshold for activation. It effectively models basic gates like AND, OR, and NOT, demonstrating the foundational concept behind neural network computation.

CO's Covered:

CO1. Implement basic neural network models.